

LAB GUIDE WITH SOLUTIONS

Lab Guide

Modules

Ten labs with full solutions · approx. 6 hours

Elixir Training Programme

Elixir · modules

Session 6 · Instructor-led · Elixir 1.20.4 on Erlang/OTP 29

How to use this guide

Ten labs, in order, roughly six hours including discussion. Each lab states a goal, gives you the tasks, and then gives the full solution with the real output it produced on the training machine.

Functions and macros are DECLARED in these labs and explained in none of them — those are sessions 7 and 8. Everything here is about the container: how you declare a module, what may go inside one, how one module is allowed to see another, and what an attribute is for.

#	Lab	Minutes	Covers
1	Declaring a module	25	defmodule as a macro, the compile-time body
2	Names and namespaces	30	module names as atoms, nesting, Module
3	What a module body accepts	35	one of each form, public vs private
4	struct, exception, delegate	35	defstruct, defexception, defdelegate
5	alias	30	four forms, lexical scope, the silent clash
6	require and import	35	macros need one, Kernel is already there
7	use and __using__	40	the expansion, options, defoverridable
8	Attributes as data	35	constants, the frozen value, accumulate
9	Docs, contracts and hooks	40	@doc, @behaviour, @before_compile
10	Capstone: a formatter registry	45	all of it, in one small library

IMPORTANT Windows users: the launchers are `iex.bat` and `mix.bat`. In PowerShell, plain `iex` collides with PowerShell's own Invoke-Expression alias — always type `iex.bat`.

WATCH OUT Two starter files sit in `code/.module_lab.exs` carries labs 1 to 3; `format.s.exs` is the capstone. Both run as handed out — they print `:todo` where your code goes.

ERRORS ON PURPOSE Several labs ask you to compile something that FAILS. Those are not mistakes in the guide — the error message is the lesson, and it is printed in full. Read it before you read the explanation underneath.

LAB 1 · 25 MINUTES · DECLARATION

Declaring a module

GOAL Stop thinking of `defmodule` as a declaration. It is a macro call that compiles a block of ordinary code and hands you back a value.

Task 1.1 — Capture what `defmodule` returns

Assign the result of a `defmodule` to a variable and take it apart.

Solution

```
result =
  defmodule Greeter do
    def hello(name), do: "hi " <> name
  end

{tag, name, bin, last} = result

tag    = :module
name   = Greeter
binary = 1728 bytes
last   = {:hello, 1}
Greeter.hello("ada") = "hi ada"
```

WHY A four-tuple: the tag, the module's name as an atom, the compiled BEAM binary, and the value of the LAST expression in the body — here the `{:hello, 1}` that `def` returned. Nothing was declared; something was built and loaded.

Task 1.2 — Keyword or macro?

Solution

```
macro_exported?(Kernel, :defmodule, 2)  #=> true
is_atom(Greeter)                        #=> true
```

WHY `defmodule` is a two-argument macro in `Kernel`, imported into every module for you. That is why it can appear anywhere an expression can — inside a `for`, inside a comprehension, inside another function.

Task 1.3 — Prove the body runs at compile time

Put an `IO.puts` above a `def` and another below it. Add a third AFTER the module. Predict the order. Then add an attribute holding a timestamp and read it twice, twenty milliseconds apart.

Solution

```
defmodule Noisy do
  IO.puts(" 1. top of the body")
  @built_at :erlang.system_time(:millisecond)
  def built_at, do: @built_at
  IO.puts(" 2. bottom of the body")
end

IO.puts(" 3. after the module")
```

1. top of the body
2. bottom of the body
3. after the module

```
built_at() twice, 20ms apart: 1790162922535 then 1790162922535
same value? true
```

WHY Both `IO.puts` ran while the module was being compiled, in source order, and they never run again — they are not part of any function. `:erlang.system_time/1` was evaluated once, at that moment, and baked into `built_at/0`. This one fact explains most of lab 8.

Task 1.4 — Build modules in a loop

If `defmodule` is an expression, you can call it from a comprehension. Generate two modules whose names and bodies come from a list.

Solution

```
for n <- [:alpha, :beta] do
  mod = Module.concat(Generated,
    n |> Atom.to_string() |> String.capitalize())
  Module.create(mod,
    quote(do: (def name, do: unquote(n))),
    Macro.Env.location(__ENV__))
end

Generated.Alpha.name()  #=> :alpha
Generated.Beta.name()   #=> :beta
```

WHY `Module.create/3` is the explicit form of what `defmodule` does. This is how Ecto turns a schema into a module and how a router turns a list of routes into compiled clauses — no reflection at run time, just code written by code at compile time.

CHECKPOINT You can say what `defmodule` returns, and why an attribute holding `DateTime.utc_now()` is a bug.

LAB 2 · 30 MINUTES · NAMES

Names and namespaces

GOAL See through the capital letter. A module name is an atom, nesting is a prefix, and neither fact is hidden anywhere deep.

Task 2.1 — Unmask a nested name

Solution

```
defmodule Outer do
  defmodule Inner do
    def who, do: __MODULE__
  end
  def who, do: __MODULE__
end

Outer.who()           #=> Outer
Outer.Inner.who()     #=> Outer.Inner
is_atom(Outer.Inner)  #=> true
Atom.to_string(Outer.Inner)  #=> "Elixir.Outer.Inner"
Outer.Inner == :Elixir.Outer.Inner  #=> true
```

WHY `__MODULE__` is the atom the compiler is actually using. The capital letter and the dots are surface syntax; underneath there is one flat name.

Task 2.2 — Take a name apart and put it back

Solution

```
Module.split(Outer.Inner)           #=> ["Outer", "Inner"]
Module.concat(["Outer", "Inner"]) == Outer.Inner  #=> true
Module.concat(Outer, Inner)          #=> Outer.Inner
```

WHY `Module.split/1` strips the `Elixir.` prefix for you and gives back strings. `Module.concat/1` puts it back. Both are ordinary run-time functions operating on an atom.

Task 2.3 — Is Inner inside Outer?

Ask both modules what functions they have.

Solution

```
Outer.__info__(:functions)  #=> [who: 0]
Outer.Inner.__info__(:functions)  #=> [who: 0]
```

WHY Two modules, each with one function. Inner is not listed inside Outer and cannot see its private functions. Nesting bought you a prefix and an implied alias inside the outer body — that is all. On disk you get `Elixir.Outer.beam` and `Elixir.Outer.Inner.beam`.

Task 2.4 — The two concats

Call both `Module.concat/1` and `Module.safe_concat/1` on a name nothing has ever used, and watch the difference.

Solution

```
Module.concat(["Never", "Seen", "Here"])
#=> Never.Seen.Here          an atom was created

Module.safe_concat(["Also", "Never", "Seen"])
** (ArgumentError) errors were found at the given arguments:
* 1st argument: not an already existing atom
```

WHY This is session 5's denial of service in module clothing. `Module.concat/1` on a name that came from a URL, a config file or a form field creates an atom the VM will never collect. `Module.safe_concat/1` refuses. If the name did not come from your own source, use the safe one.

Task 2.5 — A name with nothing behind it

Solution

```
is_atom(:"Elixir.NoSuchModule")    #=> true
Code.ensure_loaded?(NoSuchModule)   #=> false

NoSuchModule.run()
** (UndefinedFunctionError) function NoSuchModule.run/0 is undefined
(module NoSuchModule is not available). Make sure the module name is
correct and has been specified in full (or that an alias has been defined)
```

WHY Writing the name is enough to make the atom. Whether any code answers to it is a separate question, and one that is only asked when you call it — which is why a typo in a module name is a run-time error, not a compile error.

CHECKPOINT You can explain to someone who expects packages why `MyApp.Accounts` and `MyApp.Accounts.User` are unrelated modules that merely share a prefix.

LAB 3 · 35 MINUTES · INVENTORY

What a module body accepts

GOAL Write one module containing one of everything, then ask it what survived.

Use `code/module_lab.exs` for this lab.

Task 3.1 — One of each

Build a module with a moduledoc, an attribute, a struct, a public and a private guard, a public and a private function, a public and a private macro, a delegate and a nested module.

Solution

```
defmodule Toolbox do
  @moduledoc "One of each form, as a checklist."
  @limit 10

  defstruct [:name, count: 0]

  defguard is_small(n) when is_integer(n) and n <= @limit
  defguardp is_tiny(n) when is_integer(n) and n <= 2

  def public(x), do: private(x) + @limit
  defp private(x), do: x * 2

  defmacro twice(x) do
    quote do: unquote(x) * 2
  end

  defmacro shout(x) do
    quote do: String.upcase(unquote(x))
  end

  def loud(s), do: shout(s)
  def tiny?(n), do: is_tiny(n)

  defdelegate upcase(s), to: String

  defmodule Nested do
    def here, do: :nested
  end
end
```

```
Toolbox.public(3)      = 16
Toolbox.loud("quiet")  = "QUIET"
Toolbox.tiny?(1)       = true
Toolbox.upcase("hi")   = "HI"
Toolbox.Nested.here()  = :nested
Toolbox.__struct__()   = %Toolbox{name: nil, count: 0}
```

WHY A private macro and a private guard are perfectly legal — they just have to be USED inside the module, which is what `loud/1` and `tiny?/1` are for. A private macro must also be defined before the function that calls it.

Task 3.2 — Ask what survived

Solution

```
Toolbox.__info__(:functions)
#=> [__struct__: 0, __struct__: 1, loud: 1, public: 1,
      tiny?: 1, upcase: 1]

Toolbox.__info__(:macros)
#=> [is_small: 1, twice: 1]

function_exported?(Toolbox, :public, 1)    #=> true
function_exported?(Toolbox, :private, 1)   #=> false
macro_exported?(Toolbox, :twice, 1)        #=> true
macro_exported?(Toolbox, :shout, 1)        #=> false
```

WHY The rule: `def` and `defmacro` appear, `defp` and `defmacrop` do not, and the guards split the same way. `defstruct` gave you two functions you did not write, `upcase/1` is an ordinary public function, and `Nested` is nowhere in this listing because it is a separate module.

Task 3.3 — Call a private function from outside

Predict the error before you run it. Most people get the wording wrong.

Solution

```
Toolbox.private(3)
** (UndefinedFunctionError) function Toolbox.private/1 is undefined
or private
```

WHY “undefined OR private” — the VM will not tell you which, and that is deliberate. From outside the module the two are the same situation, and a caller has no business knowing that a private function exists.

CHECKPOINT Given any `def` line, you can say whether it will show up in `__info__(:functions)`, in `__info__(:macros)`, or in neither.

LAB 4 · 35 MINUTES · DECLARING

struct, exception, delegate

GOAL Three forms that each build something bigger than they look: a struct, a raisable struct, and a function that is really someone else's.

Task 4.1 — A struct with a required key

Solution

```
defmodule Account do
  @enforce_keys [:id]
  defstruct [:id, :owner, balance: 0]
end

%Account{id: 1, owner: "ada"}
#=> %Account{id: 1, owner: "ada", balance: 0}

Account.__struct__()
#=> %Account{id: nil, owner: nil, balance: 0}

Account.__info__(:struct)
#=> %{default: nil, field: :id, required: true}
    %{default: nil, field: :owner, required: false}
    %{default: 0, field: :balance, required: false}
```

WHY A field written bare gets nil; a field written key: value gets that default. @enforce_keys does not change the default — :id is still nil in __struct__/0 — it changes whether you are allowed to leave it out.

Task 4.2 — Leave the required key out

Build the struct in a SECOND module — a struct cannot be used in the same context that defines it.

Solution

```
defmodule Forgot do
  def make, do: %Account{owner: "ada"}
end

** (ArgumentError) the following keys must also be given when
building struct Account: [:id]
    expanding struct: Account.__struct__/1
    Forgot.make/0
```

WHY Read the trace: expanding struct. This is a COMPILE-time error in the module that builds the struct, not a run-time error when make/0 is called. @enforce_keys only works on struct literals for that reason.

Task 4.3 — Two exceptions

Write one exception with a fixed message: and one that computes its message from a field.

Solution

```
defmodule InsufficientFunds do
  defexception [:needed, message: "insufficient funds"]
```

```

end

defmodule Overdrawn do
  defexception [:by]

  @impl Exception
  def message(%Overdrawn{by: n}), do: "overdrawn by #{n}"
end

raise InsufficientFunds, needed: 50
raise Overdrawn, by: 120

```

```

rescued InsufficientFunds: insufficient funds
rescued Overdrawn: overdrawn by 120

rescue e in InsufficientFunds -> e.needed    #=> 50

keys: [:__exception__, :__struct__, :message, :needed]

```

WHY `defexception` is `defstruct` plus two things: a `__exception__` key, and a default `message/1` that returns the `:message` field. Override `message/1` when the text depends on the data, and keep the fields so a caller can match on them rather than parse your sentence.

Task 4.4 — Delegate three ways

Solution

```

defmodule Text do
  defdelegate upcase(s), to: String
  defdelegate shout(s), to: String, as: :upcase
  defdelegate length(s), to: String, as: :length
end

Text.upcase("hi")           #=> "HI"
Text.shout("hi")           #=> "HI"
Text.length("hi")          #=> 2
Text.__info__(:functions)  #=> [length: 1, shout: 1, upcase: 1]

```

WHY Each one is a real, public, compiled function — they are in `__info__(:functions)` like anything else. Note `length/1: Text.length/1` worked here where lab 6 will show the same name failing, because `defdelegate` introduces the name rather than defining it with `def`.

CHECKPOINT You can write an exception whose message is computed, and you can say why `@enforce_keys` is a compile-time check and not a run-time one.

LAB 5 · 30 MINUTES · DIRECTIVES

alias

GOAL The weakest of the four directives, and the one with a trap nobody warns you about.

Task 5.1 — Four ways to write it

Solution

```
defmodule MyApp.Accounts.User do
  def role, do: :admin
end

alias MyApp.Accounts.User          # -> User
alias MyApp.Accounts.User, as: U   # -> U
alias MyApp.Accounts.{User}        # -> User
alias :lists, as: L                # an Erlang module

User.role()      #=> :admin
U.role()         #=> :admin
L.reverse([1, 2, 3]) #=> [3, 2, 1]
```

WHY With no `as:` the alias is the LAST segment — `User`, not `Accounts`. The brace form is for several modules under one prefix. And `as:` is the only way to give an Erlang module a capitalised name, because `alias :lists` is not valid syntax.

Task 5.2 — Prove it is compile-time only

Return the aliased name itself from a function and see what comes out at run time.

Solution

```
defmodule Proof do
  alias String.Chars, as: C
  def f, do: C
end

Proof.f() #=> String.Chars
```

WHY The short name is gone by the time the module is compiled. An alias creates no dependency, loads no code, and costs nothing at run time — which is why it is the directive to reach for first.

Task 5.3 — Lexical scope

Solution

```
defmodule Scoped do
  def inside do
    alias Enum, as: E
    E.count([1, 2, 3])
  end

  def outside, do: Enum.count([1, 2, 3]) # E means nothing here
end
```

```
Scoped.inside()    #=> 3  
Scoped.outside()   #=> 3
```

WHY An alias covers the block it was written in. At the top of a module body that is the whole module; inside a function it is that function. All four directives behave this way.

Task 5.4 — Two aliases, one short name

Alias two different modules to the same short name. Predict whether you get a warning, an error, or neither.

Solution

```
defmodule Clash do  
  alias Map.Helper  
  alias List.Helper  
  def which, do: Helper.who()  
end  
  
Clash.which()    #=> :from_list  
  
no warning. no error.
```

WHY Neither. The second alias silently replaces the first. In a long module body, with the two lines fifty lines apart, this is genuinely hard to see — which is why a name you alias twice deserves an explicit `as` on one of them.

CHECKPOINT You can say what a `alias A.B.C` aliases to, and you can name the one situation where two aliases go wrong without telling you.

LAB 6 · 35 MINUTES · DIRECTIVES

require and import

GOAL One directive exists only for macros; the other is the reason you cannot call your function `length/1`.

Task 6.1 — A macro without require

Call `Integer.is_odd/1` from a module with no `require`, and read the warning.

Solution

```
defmodule NoRequire do
  def odd?(x), do: Integer.is_odd(x)
end

warning: Integer.is_odd/1 is undefined or private. However, there is
a macro with the same name and arity. Be sure to require Integer if
you intend to invoke this macro

# with the require, it simply works:
defmodule WithRequire do
  require Integer
  def odd?(x), do: Integer.is_odd(x)
end

WithRequire.odd?(3)    #=> true
WithRequire.odd?(4)    #=> false
```

WHY The compiler can see that a macro of that name exists and tells you exactly what to add. A macro must be EXPANDED before your module finishes compiling, so the compiler has to load `Integer` first — `require` is you saying it may.

Task 6.2 — A function never needs it

Solution

```
defmodule NoRequireNeeded do
  def count(l), do: Enum.count(l)
end

NoRequireNeeded.count([1, 2, 3])    #=> 3
no require, no warning
```

WHY A function call is resolved at run time, so the compiler does not need the module in front of it. If you have been writing `require` out of habit, this is the lab that stops you.

Task 6.3 — The four import forms

Solution

```
defmodule Forms do
  import List, only: [first: 1, last: 1]    # a named list
  import String, only: [upcase: 1]
  import Integer, only: :macros             # or :functions, :sigils
  import Enum, except: [count: 1, sum: 1]
```

```
def ends(l), do: {first(l), last(l)}
def shout(s), do: upcase(s)
def odd?(n), do: is_odd(n)
def doubled(l), do: map(l, &(&1 * 2))
end

Forms.ends([1, 2, 3])    #=> {1, 3}
Forms.shout("hi")        #=> "HI"
Forms.odd?(3)            #=> true
Forms.doubled([1, 2])    #=> [2, 4]
```

WHY `import Integer, only: :macros` brought in `is_odd/1` AND the `require` it needs — `import` implies `require`. Note also that two imports only collide when you actually CALL an ambiguous name; `List` and `String` both export `first/1`, and nothing complains until you write it unqualified.

Task 6.4 — Collide with Kernel, then fix it

Try to define your own `length/1`.

Solution

```
defmodule Collide do
  def length(x), do: {:mine, x}
  def call, do: length("abc")
end

** (CompileError)
error: imported Kernel.length/1 conflicts with local function
```

```
defmodule Fixed do
  import Kernel, except: [length: 1]
  def length(x), do: {:mine, x}
  def call, do: length("abc")
  def kernel_one, do: Kernel.length([1, 2, 3])
end

Fixed.call()          #=> {:mine, "abc"}
Fixed.kernel_one()    #=> 3
```

WHY Every module starts with `import Kernel` and `require Kernel` already done for you — that is why `+`, `if`, `def` and `raise` work with no prefix. `except:` is how you take one name back, and the full name still works.

CHECKPOINT You can state the one condition that makes `require` necessary, and you can fix a `Kernel` conflict without renaming your function.

LAB 7 · 40 MINUTES · DIRECTIVES

use and __using__

GOAL Write the directive that writes code. This is the whole of what `GenServer`, `ExUnit.Case` and `Ecto.Schema` do.

Task 7.1 — What `use` expands to

Expand it once with `Macro.expand_once/2` and read the result.

Solution

```
quote do
  use SomeModule, some: :option
end
|> Macro.expand_once(__ENV__)
|> Macro.to_string()

(
  require SomeModule
  SomeModule.__using__(some: :option)
)
```

WHY Two lines. That is the entire definition of `use`. Everything a library appears to do to your module is done by the macro `__using__/1`, which you can read.

Task 7.2 — Write one

Write `Greetable` whose `__using__/1` injects a `greet/1` function. Take the greeting from an option, with a default, and let the using module replace the function.

Solution

```
defmodule Greetable do
  defmacro __using__(opts) do
    greeting = Keyword.get(opts, :greeting, "hello")

    quote do
      def greet(name), do: unquote(greeting) <> " " <> name
      def greeting_from, do: unquote(__MODULE__)
      defoverridable greet: 1
    end
  end
end
```

WHY `opts` is read at compile time, in the macro, so `greeting` is a plain string by the time `unquote/1` embeds it. `__MODULE__` inside the quote refers to `Greetable` — the module that DEFINED the macro, not the one using it.

Task 7.3 — Use it three ways

Solution

```
defmodule Plain do
  use Greetable
end
```

```
defmodule Hindi do
  use Greetable, greeting: "namaste"
end

defmodule Shouty do
  use Greetable, greeting: "oi"
  def greet(name), do: String.upcase(super(name))
end

Plain.greet("ada")      #=> "hello ada"
Hindi.greet("ravi")     #=> "namaste ravi"
Shouty.greet("kiran")   #=> "OI KIRAN"
Plain.greeting_from()   #=> Greetable
Plain.__info__(:functions) #=> [greet: 1, greeting_from: 0]
```

WHY Plain did not write those two functions and they are hers all the same — `__info__` lists them exactly like hand-written ones. `super/1` calls the injected version, so Shouty got the option AND the override.

Task 7.4 — Remove defoverridable and watch it break

Solution

```
defmodule NoOverride do
  defmacro __using__(_) do
    quote do
      def greet(name), do: "hello " <> name
    end
  end
end

defmodule Breaks do
  use NoOverride
  def greet(name), do: String.upcase(super(name))
end

** (CompileError)
error: no super defined for greet/1 in module Breaks.
Overridable functions available are:
```

WHY The list of overridable functions is empty, and the compiler prints it empty. `defoverridable` is what a library author writes to say “you may replace this”; without it, redefining the function is a clause conflict and `super/1` has nothing to call.

CHECKPOINT You can read a library's `__using__/1` and predict exactly which functions your module will end up with.

LAB 8 · 35 MINUTES · @ATTRIBUTES

Attributes as data

GOAL Compile-time constants, the value that freezes, and the two registrations that change what an attribute does.

Task 8.1 — Constants, including in a guard

Solution

```
defmodule Limits do
  @limit 10
  @message "over the limit"
  @primes [2, 3, 5, 7]

  def limit, do: @limit
  def primes, do: @primes
  def check(n) when n > @limit, do: @message
  def check(_), do: :ok
end

Limits.limit()      #=> 10
Limits.primes()     #=> [2, 3, 5, 7]
Limits.check(11)    #=> "over the limit"
Limits.check(3)     #=> :ok
```

WHY An attribute is legal in a guard because it is INLINED — by the time the guard is compiled, `@limit` is the literal 10. A function call would not be allowed there; a constant is not a call.

Task 8.2 — Read at the point of use

Solution

```
defmodule Rebind do
  @v 1
  def a, do: @v
  @v 2
  def b, do: @v
end

Rebind.a()  #=> 1
Rebind.b()  #=> 2
```

WHY Each `def` took the value the attribute had at the moment the compiler reached it. An attribute is not a constant in the usual sense — it is a compile-time variable you may reassign, and it is read where it is written.

Task 8.3 — Read one you never set

Solution

```
defmodule Missing do
  def f, do: @never_set
end

warning: undefined module attribute @never_set, please remove access
```

to `@never_set` or explicitly set it before access

WHY A warning, not an error, and `f/0` returns `nil`. A typo in an attribute name is therefore a silent `nil` at run time unless you read your warnings.

Task 8.4 — The frozen value

Put a timestamp in an attribute and a timestamp in a function, and compare them twenty-five milliseconds apart.

Solution

```
defmodule Frozen do
  @compiled_at :erlang.system_time(:millisecond)
  def compiled_at, do: @compiled_at
  def now, do: :erlang.system_time(:millisecond)
end

compiled_at() 25ms apart: 1790162996526 then 1790162996526
same? true
now() moves: true
```

WHY `DateTime.utc_now()` in an attribute records BUILD time. In a release compiled on a build server weeks ago, that is what every request will report. `@version Mix.Project.config()[ :version]` is the correct use of the same mechanism — there you WANT the build's value.

Task 8.5 — accumulate and persist

Solution

```
defmodule Steps do
  Module.register_attribute(__MODULE__, :step, accumulate: true)
  Module.register_attribute(__MODULE__, :owner, persist: true)

  @owner "platform-team"
  @step :first
  @step :second
  @step :third
  @plain "not persisted"

  def steps, do: @step
  def plain, do: @plain
end

Steps.steps()  #=> [:third, :second, :first]    newest first
Steps.plain()  #=> "not persisted"

Steps.__info__(:attributes)
#=> [vs: [198053467610784376875202410031...],
     owner: ["platform-team"]]
```

WHY Two independent choices. `accumulate: true` makes repeated writes build a list instead of overwriting — newest first, so reverse it if order matters. `persist: true` puts the attribute in the `.beam` where `__info__(:attributes)` can read it back; `@step` and `@plain` were both inlined and then discarded.

CHECKPOINT You can explain why `@now DateTime.utc_now()` is a bug and `@version ...` is not.

LAB 9 · 40 MINUTES · @ATTRIBUTES

Docs, contracts and hooks

GOAL The attributes the compiler and the tooling read, rather than your own code.

Task 9.1 — Write docs, then read them back

Compile the module to a real `.beam` with `elixirc`, then ask `Code.fetch_docs/1` what it recorded.

Solution

```
defmodule Documented do
  @moduledoc "A module that documents itself."

  @typedoc "A single digit."
  @type digit :: 0..9

  @doc "Adds two numbers."
  @spec add(number(), number()) :: number()
  def add(a, b), do: a + b

  @doc false
  def internal, do: :hidden

  def undocumented, do: :no_doc
end
```

```
elixirc -o out documented.ex
Code.fetch_docs(Documented)

language = :elixir
@moduledoc = "A module that documents itself."

function add/2      doc="Adds two numbers."
function internal/0  doc=:hidden      <- @doc false
function undocumented/0 doc=:none      <- no @doc at all
type digit/0        doc="A single digit."
```

WHY Three different states, and the difference matters: `:hidden` means the function is public but deliberately undocumented, while `:none` means nobody wrote one. Tools treat them differently — `@doc false` hides a function, no `@doc` just leaves it blank.

Task 9.2 — Now do it in a script

Solution

```
defmodule NoBeam do
  @moduledoc "never written to disk"
  def f, do: :ok
end

Code.fetch_docs(NoBeam) #=> {:error, :module_not_found}
```

WHY Docs live in a chunk of the .beam FILE, and a script never writes one. This is the answer to “why does h tell me nothing about the module I just defined in IEx?” — the module is loaded, the file does not exist.

Task 9.3 — Document a private function

Solution

```
defmodule PrivDoc do
  @doc "this will not survive"
  defp helper(x), do: x
  def call(x), do: helper(x)
end

warning: defp helper/1 is private, @doc attribute is always discarded
for private functions/macros/types
```

WHY Use an ordinary # comment for a private function. @doc is for the public surface, and the compiler will not let you pretend otherwise.

Task 9.4 — A behaviour, half implemented

Solution

```
defmodule Formatter do
  @callback format(term()) :: String.t()
  @callback extension() :: String.t()
end

defmodule JsonFormatter do
  @behaviour Formatter

  @impl Formatter
  def format(term), do: inspect(term)
end

warning: function extension/0 required by behaviour Formatter is not
implemented (in module JsonFormatter)
```

```
# add the missing callback:
@impl Formatter
def extension, do: "json"

JsonFormatter.format(%{a: 1})  #=> "%{a: 1}"
JsonFormatter.extension()      #=> "json"

JsonFormatter.__info__(:attributes)[:behaviour]  #=> [Formatter]
```

WHY @callback declares the contract, @behaviour adopts it, @impl marks which callback a function implements. The adopted behaviours are persisted, so __info__(:attributes) can read them back — lab 10 uses exactly that to audit a registry.

Task 9.5 — The three hooks

Solution

```
defmodule Hooks do
  defmacro __before_compile__(_env) do
    quote do: (def injected, do: :added_last)
  end

  def __after_compile__(env, _bytecode) do
    IO.puts(" __after_compile__ for #{inspect(env.module)}")
  end

  def __on_definition__(_env, kind, name, args, _guards, _body) do
    IO.puts(" __on_definition__ #{kind} #{name}/#{length(args)}")
  end
end

defmodule Watched do
  @before_compile Hooks
  @after_compile Hooks
  @on_definition Hooks

  def a, do: 1
  defp b, do: 2
  def c, do: b()
end
```

```
__on_definition__ def a/0
__on_definition__ defp b/0
__on_definition__ def c/0
__on_definition__ def injected/0
__after_compile__ for Watched

Watched.injected()  #=> :added_last
```

WHY Read the order. `__on_definition__` fired for the private function too, and for `injected/0` — the function that `@before_compile` added AFTER the body finished. `__after_compile__` ran last of all, and gets the bytecode, by which point it is too late to change anything.

CHECKPOINT Given a module that uses `@before_compile`, you can say which of its functions you wrote and which the hook added.

LAB 10 · 45 MINUTES · ALL OF IT

Capstone: a formatter registry

GOAL A behaviour, a `__using__` that implements half of it, three formatters and a compile-time registry — the shape of most small Elixir libraries.

Start from `code/formats.exs`. The contract and the sample data are given; `Format.__using__/1`, the formatters' bodies and the whole of `Formats` are yours.

Task 10.1 — The behaviour and its `__using__`

Solution

```
defmodule Format do
  @callback format(term()) :: String.t()
  @callback extension() :: String.t()

  defmacro __using__(opts) do
    ext = Keyword.fetch!(opts, :extension)

    quote do
      @behaviour Format

      @impl Format
      def extension, do: unquote(ext)

      @impl Format
      def format(term), do: inspect(term)

      defoverridable format: 1
    end
  end
end
```

WHY `Keyword.fetch!`, not `Keyword.get`. A formatter with no extension is a bug, and you want it at compile time — task 10.5 shows what that looks like. Note that `format/1` is injected and then immediately made overridable: the default is real, working code, not a stub that raises.

Task 10.2 — Three formatters, two of which override

Solution

```
defmodule Format.Json do
  use Format, extension: "json"

  @impl Format
  def format(term) do
    term
    |> Enum.map(fn {k, v} -> "\"#{k}\"": #{inspect(v)}" end)
    |> Enum.join(", ")
    |> then(&("{ " <> &1 <> "}")
  end
end
```

```
defmodule Format.Csv do
  use Format, extension: "csv"

  @impl Format
  def format(term) do
    term |> Enum.map(fn {_k, v} -> to_string(v) end) |> Enum.join(",")
  end
end

defmodule Format.Inspect do
  use Format, extension: "txt"
end
# keeps the default format/1
```

WHY `Format.Inspect` is three lines and a complete, conforming formatter. That is the point of putting a working default in `__using__` rather than a stub.

Task 10.3 — The registry

Solution

```
defmodule Formats do
  @formatters %{
    "json" => Format.Json,
    "csv" => Format.Csv,
    "txt" => Format.Inspect
  }

  def known, do: Map.keys(@formatters)

  def fetch(ext) when is_binary(ext) do
    case Map.fetch(@formatters, ext) do
      {:ok, mod} -> {:ok, mod}
      :error -> {:error, {:unknown_format, ext}}
    end
  end

  def fetch(_, do: {:error, :bad_format})

  def render(data, ext) do
    with {:ok, mod} <- fetch(ext) do
      {:ok, mod.format(data)}
    end
  end
end
```

```
data = [name: "Ada", role: "admin"]

known()  #=> ["csv", "json", "txt"]

render(data, "json")
#=> {:ok, "{\"name\": \"Ada\", \"role\": \"admin\"}"}
render(data, "csv")  #=> {:ok, "Ada,admin"}
render(data, "txt")
#=> {:ok, "[name: \"Ada\", role: \"admin\"]"}
render(data, "yaml") #=> {:error, {:unknown_format, "yaml"}}
```

```
render(data, :json)  #=> {:error, :bad_format}
```

WHY `@formatters` is a module attribute, so the map is built once at compile time and the lookup is a lookup, not a scan. And notice what the registry does NOT do: it never turns `"yaml"` into an atom. The extension stays a string all the way through — session 5's rule, applied to module names.

Task 10.4 — Audit the registry

Write `Formats.audit/0` reporting, for each formatter, its module, its extension, the behaviours it adopted, and whether it overrode the default `format/1`.

Solution

```
def audit do
  for {ext, mod} <- @formatters, into: %{} do
    {ext,
     %{
       module: mod,
       extension: mod.extension(),
       behaviours: mod.__info__(:attributes)[:behaviour],
       override_format?: mod.format([{:a, 1}]) != inspect([{:a, 1}])
     }}
  end
end
```

csv	Format.Csv	ext="csv"	behaviours=[Format]	override=true
json	Format.Json	ext="json"	behaviours=[Format]	override=true
txt	Format.Inspect	ext="txt"	behaviours=[Format]	override=false

WHY Every column here came from a different mechanism this session covered: the extension from an option threaded through `__using__`, the behaviour list from a persisted attribute, and the override flag from comparing the module's `format/1` against the default the macro injected.

Task 10.5 — Forget the option

Write a formatter that says use `Format` with no `extension:`, and read where the error points.

Solution

```
defmodule Format.Broken do
  use Format
end

** (KeyError) key :extension not found in:

[]

(elixir 1.20.4) lib/keyword.ex:618: Keyword.fetch!/2
expanding macro: Format.__using__/1
Format.Broken (module)
expanding macro: Kernel.use/1
Format.Broken (module)
```

WHY Read the trace from the bottom: `Kernel.use/1`, then `Format.__using__/1`, then `Keyword.fetch!/2`. A macro's failure is reported as a compile-time expansion failure, with the whole chain shown. This is exactly what you wanted — the alternative, `Keyword.get`, ships a formatter that writes files with no extension.

Task 10.6 — Extend it (if there is time)

Pick one: add an `encode/1` callback and have `render/2` call it after `format/1`; make `@formatters` build itself from `Module.concat` instead of a hand-written map; or add a `@after_compile` hook that refuses to compile a formatter whose extension is already taken.

WATCH OUT The third one is the interesting exercise, and the honest answer is that it is harder than it looks: `@after_compile` runs per module, and the module being compiled cannot see the others. A registry that validates across modules needs a real compile-time store — which is why most libraries do the check in the registry module instead, where all the formatters are already listed.

CHECKPOINT Your registry answers with a tagged tuple for every input, creates no atoms from user data, and `audit/0` tells you which formatters kept the default `format/1`.

Where this goes next

You now have the container. The next two sessions fill it: session 7 is functions — clauses, arity, defaults, captures and closures — and session 8 is macros, which is where `defmacro` and `__using__` stop being something you copy from a library and start being something you write.

If you remember one thing per lab	It is this
1 · Declaring	the body is code, and it runs while compiling
2 · Names	a module name is an atom; nesting is only a prefix
3 · The body	<code>def</code> and <code>defmacro</code> are listed; <code>defp</code> and <code>defmacrop</code> are not
4 · struct / exception	<code>@enforce_keys</code> fails at compile time, in the caller
5 · alias	compile-time renaming, and the second one silently wins
6 · require / import	only macros need <code>require</code> ; <code>Kernel</code> is already imported
7 · use	<code>require Foo</code> , then <code>Foo.__using__(opts)</code> — read it before you use it
8 · Attributes	inlined where used, evaluated once, gone by run time
9 · Docs and contracts	docs live in the <code>.beam</code> file; <code>@behaviour</code> is checked
10 · Capstone	a behaviour, a <code>__using__</code> and a compile-time map is a library

The one-line rule

Reach for the weakest thing that works: `nothing`, then `alias`, then `require`, then `import`, and `use` only when the library tells you to. Everything stronger hides where a name came from.

MEASURED Every output in this guide was produced by `../verify/probe_labs6.exs` on Elixir 1.20.4 / Erlang/OTP 29. Most solutions are compiled in a child VM, so the warnings and errors printed here are the ones your compiler will print. If yours disagree, your machine is right — say so and work out why.